

Preface

How did you come to find this book? Did you see an ad for it on a website? Maybe a friend or mentor suggested it; or perhaps you saw a post on social media that referenced it. Could it be that you found it sitting on a shelf in a bookstore—a bookstore that your trusty maps app led you to? However you came to find it, you've almost certainly come to this book via a recommendation system.

Implementing and designing systems that provide suggestions to users is among the most popular and most essential applications of machine learning (ML) to any business. Whether you want to help your users find the best clothing to match their tastes, the most appealing items to buy from an online store, videos to enrich and entertain them, maximally engaging content to surface from their networks, or the news highlights they need to know on that day, recommendation systems provide the way.

Modern recommendation system designs are as diverse as the domains they serve. These systems consist of the computer software architectures to implement and execute product goals, in addition to the algorithmic components of ranking. Methods for ranking recommendations can come from traditional statistical learning algorithms, linear-algebraic inspirations, geometric considerations, and, of course, gradient-based methods. Just as the algorithmic methods are diverse, so too are the modeling and evaluation considerations for recommending: personalized ranking, search recommendations, sequence modeling, and the scoring for all of these are now need-to-know for the ML engineer working with recommendation systems.

NOTE

The abbreviation RecSys is often used by practitioners to describe the field of recommendation systems. Therefore, in this book, we use RecSys when referring to the field, and recommendation system when referring to what we build.

If you're an ML practitioner, you are probably aware of recommendation systems, and you may know one or two of the simplest modeling approaches and be able to speak intelligently about the relevant data structures and model architectures; however, RecSys frequently falls outside the core curriculum of data science and ML. Many senior data scientists with years of experience in the industry know little about actually building a recommendation system and may feel intimidated when the topic comes up. Despite drawing on similar foundations and skills as other ML problems, RecSys has a vibrant community with a fast-moving focus that can make it easy to relegate building recommendation systems to *other* data scientists who have already invested the time, or are willing to stay on top of the latest information.

The reason this book exists, is to break through those perceived barriers. Understanding recommendation systems at a practical level is not only useful for business cases requiring content to be served to users, but the underlying ideas of RecSys often bridge gaps between an incredibly diverse set of other types of ML. Take, for example, an article recommendation system that may utilize natural language processing (NLP) to find representations of the articles, sequential modeling to promote longer engagement, and contextual components to allow user queries to guide results. If you're approaching the field from a purely academic interest, no matter what aspects of mathematics you're interested in, sooner or later, there appears a link or application in RecSys!

Finally, if connections to other fields, applications of nearly all of mathematics, or the obvious business utility *aren't* enough to get you interested in RecSys, the stunning cutting-edge technology might: RecSys is at and beyond the forefront of ML at all times. One benefit of having obvious revenue impact is that companies and practitioners need to always be pushing the boundaries of what is possible and how they go about it. The

most advanced deep learning architectures and best code infrastructures are brought to bear on this field. That’s hardly a surprise when you consider that at the heart of four of the five letters in FAANG—which stands for Meta (formerly Facebook), Apple, Amazon, Netflix, and Google—lies one or many recommendation systems.¹

As a practitioner, you’ll need to understand how to do the following:

- Take your data and business problem and frame it as a RecSys problem
- Identify the essential data to get started building a RecSys
- Determine the appropriate models for your RecSys problem and how should you evaluate them.
- Implement, train, test, and deploy the aforementioned models
- Track metrics to ensure that your system is working as planned
- Incrementally improve your system as you learn more about your users, products, and business case

This book illustrates the core concepts and examples necessary to complete these steps, whatever the industry or scale. We’ll guide you through the math, ideas, and implementation details for building recommendation systems—whether it’s your first or your fiftieth. We’ll show you how to build these systems with Python and JAX.

If you’re not yet familiar, JAX is a Python framework from Google that seeks to make autodifferentiation and functional programming paradigms first-class objects. Additionally, it uses a NumPy API style especially convenient for ML practitioners from a variety of backgrounds.

We will show code examples and architecture models that capture the essential concepts necessary and provide the way to scale these systems to production applications.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

`Constant width italic`

Shows text that should be replaced with user-supplied values or by values determined by context.

TIP

This element signifies a tip or suggestion.

NOTE

This element signifies a general note.

WARNING

This element indicates a warning or caution.

Using Code Examples

The included code snippets reference notebooks that will run on moderate-size and, in most cases, free resources. To facilitate easy experimentation and exploration we provide the code via Google Colab notebooks.

Supplemental material (code examples, exercises, etc.) is available for download at [ESRecsys on GitHub](#).

If you have a technical question or a problem using the code examples, please send email to bookquestions@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Building Recommendation Systems in Python and JAX* by Bryan Bischof and Hector Yee. Copyright 2024 Bryan Bischof and Resonant Intelligence LLC, 978-1-492-09799-0.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning

NOTE

For more than 40 years, [O'Reilly Media](#) has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live

training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit

<http://oreilly.com.ezproxy.sfpl.org>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-889-8969 (in the United States or Canada)

707-829-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

<https://www-oreilly-com.ezproxy.sfpl.org/about/contact.html>

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at

https://oreil.ly/build_rec_sys_python_jax.

For news and information about our books and courses, visit

<https://oreilly-com.ezproxy.sfpl.org>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>

Follow us on Twitter: <https://twitter.com/oreillymedia>

Watch us on YouTube: <https://youtube.com/oreillymedia>

Acknowledgments

Hector would like to thank his husband, Donald, for his loving support during the writing of this book and for the snacks his sister Serena sends all the time. He would also like to dedicate this book to his relatives who have passed. A big thank you goes to the Google reviewers Ed Chi, Courtney Hohne, Sally Goldman, Richa Nigam, Mingliang Jiang, and Anselm Levskaya. Thanks to Bryan Hughes for reviewing the Wikipedia code.

Bryan would like to thank his colleagues from Stitch Fix, where he learned many of the key ideas in this book—in particular, Ian Horn’s patient guidance on transfer learning, Dr. Molly Davies’s mentorship on experimentation and effect estimates, Mark Weiss’s deep partnership on understanding the relationship between availability and recommendations, Dr. Reza Sohrabi’s introduction to transformers, Dr. Xi Chen’s encouragement on GNNs for recs, and Dr. Leland McInnes for his careful advice on dimension reduction and approximate nearest neighbors. Bryan benefited a lot from conversations with Dr. Natalia Gardiol, Dr. Daniel Fleischman, Dr. Andrew Ho, Jason Liu, Dr. Dan Marthaler, Dr. Chris Moody, Oz Raza, Dr. Anna Schneider, Ujjwal Sarin, Agnieszka Szefer, Dr. Daniel Tasse, Diyang Tang, Zach Winston, and others he has almost certainly forgotten. Outside of his incredible Stitch Fix colleagues, he especially wants to thank Dr. Eric Bunch, Dr. Lee Goerl, Dr. Will Chernoff, Leo Rosenberg, and Janu Verma for collaboration over the years. Dr. Brian Amadio as an excellent colleague and originally suggested that he write this book. Dr. Even Oldridge for encouraging him to actually try it. Eugene Yan and Karl Higley—neither of whom he’s met but has been significantly inspired by. He’d like to thank Dr. Zhongzhu Lin and Dr. Alexander Rosenberg, who both had formative impacts on his career. Cianna Salvatora, who assisted in early literature review, and Valentina Besprozvannykh, who greatly assisted in reading early draft notes and providing guidance.

Both authors thank Tobias Zwingmann, Ted Dunning, Vicki Boykis, Eric Schles, Shaked Zychlinski, and Will Kurt, who spend much time giving

careful technical feedback on book manuscripts—without which this book would have been incomprehensible. Rebecca Novack, who harangued us into signing up for this project. And Jill Leonard, who removed nearly 100 erroneous instances of the word *utilize* from the manuscript, and who offered an incredible amount of patient partnership on the book text.

¹ Some may quibble that Apple also has core recommendation systems at the heart of its company. While it's certainly true that the App Store forms a crucial strategic product for the company, we remain conservative in our four-out-of-five assessment and say that recommendation systems are not Apple's primary revenue-generating capability.